Programming 2

Tutorial 9

Exercise 1 (required)

Build a utility library `XFile` with `read()` and `write()` functions allowing reading and writing binary files. Utilize the above library to copy one file to another.

GUIDELINES:

- Create the `XFile` class

```java
public class XFile {
    /**
     * Read binary file
     * @param path the path of the file to read
     * @return the read data
     * @throws IOException if an error occurs while reading the file
     */
    public static byte[] read(String path) {
        // code implementation
        return new byte[0];
    }
    /**
     * Write binary file
     * @param path the path of the file to write
     * @param data the data to write to the file
     * @throws IOException if an error occurs while writing the file
     */
    public static void write(String path, byte[] data) {
        // code implementation
    }
```

- Write code for the `read()` function

```java
try {
    FileInputStream fis = new FileInputStream(path);
    int n = fis.available();
    byte[] data = new byte[n];
    fis.read(data);
    fis.close();
    return data;
} catch (Exception e) {
    throw new RuntimeException(e);
}
```

- Write code for the `write()` function

```java
try {
    FileOutputStream fos = new FileOutputStream(path);
    fos.write(data);
    fos.close();
} catch (Exception e) {
    throw new RuntimeException(e);
}
```

- Create the `XFileDemo` class containing the `main()` method and utilize the `XFile` library as follows:

```java
public static void main(String[] args) {
    byte[] data = XFile.read( path: "c:/temp/a.gif");
    XFile.write( path: "c:/temp/b.gif", data);
}
```

Exercise 2 (required)

Extend the `XFile` library with two functions allowing reading and writing objects from/to a file.

```java
/**
 * Read object file
 * @param path the path of the file to read
 * @return the read object
 * @throws IOException if an error occurs while reading the file
 */
public static Object readObject(String path) {
    // code implementation
    return null;
}

/**
 * Write object file
 * @param path the path of the file to write
 * @param object the object to write to the file
 * @throws IOException if an error occurs while writing the file
 */
public static void writeObject(String path, Object object) {
    // code implementation
}
```

- Write code for the `readObject()` function

```java
try {
    ObjectInputStream ois = new ObjectInputStream(new FileInputStream(path));
    Object object = ois.readObject();
    ois.close();
    return object;
} catch (Exception e) {
    throw new RuntimeException(e);
}
```

- Write code for the `writeObject()` function

```java
try {
    ObjectOutputStream oos = new ObjectOutputStream(new FileOutputStream(path));
    oos.writeObject(object);
    oos.close();
} catch (Exception e) {
    throw new RuntimeException(e);
}
```

- Utilize the `readObject()` and `writeObject()` functions to read and write `List<Student>`. Note that the `Student` class must implement the `Serializable` interface.

```
public class Student implements Serializable {
    // Student class definition
}
```

- In the main method:

```
List<Student> list = new ArrayList<Student>();
list.add(new Student("Tuấn", 5, "CNTT"));
list.add(new Student("Cường", 7.5, "TKW"));
list.add(new Student("Hạnh", 8.5, "CNTT"));

XFile.writeObject("C:/temp/students.dat", list);
```

Exercise 3 (required)

Implement a Java class `Matrix` to represent bidimensional matrices of real numbers. The class should provide the following methods:

- `Matrix(int n, int m)` : a constructor that creates a matrix of size $n \times m$, with all values initially set to 0;
- `void save(String filename)` : saves the content of the matrix to the file specified by `filename`;
- `static Matrix read(String filename)` : reads the matrix data from the file specified by `filename`, creates the matrix, and returns it;
- `Matrix sum(Matrix m)` : returns a matrix that is the sum of the current matrix and matrix m, if the two matrices have the same dimensions, and `null` otherwise;
- `Matrix product(Matrix m)` : returns the matrix that is the product of the current one and m, if the two matrices have compatible dimensions, and `null` otherwise.

Exercise 4 (required)

Implement a class `IOFile` that that provides basic text file operations. The class should have a constructor with one parameter of type `String`, representing the name of the file to operate on, and should provide the following methods:

- `int countLines()` : returns the number of lines in the file;
- `void write(OutputStream os)` : writes the file content to the given `OutputStream`;
- `void print()` : prints the file content to the console;
- `void copy(String filename)` : copies the content of the file to the file specified by `filename`;
- `void delete()` : deletes the file.